

Based on the architectural diagram you provided, transitioning from the traditional AWS setup (EC2 Airflow -> S3 -> Glue -> Redshift -> Reverse ETL -> Postgres) to a Databricks Lakebase serverless architecture significantly simplifies the data flow. It eliminates redundant data movement and infrastructure management.

Here is a detailed, modern implementation plan you can hand over to Claude Code to build this system using best-practice serverless patterns:

Phase 1: Serverless Infrastructure as Code (IaC)

Instruct Claude Code to use the official Databricks Terraform Provider (`databricks/databricks`) to provision the cloud resources. This replaces the manual provisioning of EC2, Redshift, and RDS:

- **Databricks Workspace & Unity Catalog:** Provision the core workspace and enable Unity Catalog to provide unified governance across both analytical data and the operational database.
- **Lakebase Autoscaling Project:** Have the assistant define a Lakebase Autoscaling project. This Provisions a serverless PostgreSQL 17 environment where compute automatically scales based on demand and can scale to zero when idle.
- **Databricks App Resource:** Instruct Claude to define a `databricks_app` resource. This provides the serverless runtime environment to host your downstream Soccermetrics Streamlit application.

Phase 2: Serverless Data Ingestion (Replacing EC2 Airflow)

The original diagram relies on a "DataLab VM AWS EC2" to run Apache Airflow. To adopt a true serverless approach:

- Instruct Claude to write Python ingestion scripts using the `statsbombpy` library and standard API requests to fetch the open-source StatsBomb, Metrica, and Wyscout datasets.
- Instead of maintaining EC2 instances, ask the assistant to deploy these scripts as **Databricks Workflows** utilizing Serverless Compute. Configure the workflows to run on a schedule to fetch the data and land the raw JSON/CSV files directly into your Unity Catalog (the Bronze layer).

Phase 3: Transformation via Lakehouse & dbt (Replacing Glue & Redshift)

Instead of extracting data with AWS Glue and pushing it into an isolated Redshift warehouse for modeling:

- **The dbt Setup:** Instruct the assistant to configure a dbt project utilizing the

dbt-databricks adapter.

- **Serverless Transformations:** Have Claude write dbt SQL models that execute on Databricks Serverless SQL warehouses. These models will flatten the deeply nested JSON event structures and calculate your Soccermetrics metrics (like pass networks or Expected Threat), saving the final curated outputs as Delta tables (the Gold layer).

Phase 4: Zero-ETL Synchronization (Replacing Reverse ETL)

In the original diagram, the second LOAD step requires moving data from Redshift to Postgres (Reverse ETL) to serve the frontend application without latency. Lakebase eliminates this pipeline:

- Instruct Claude to utilize the **Databricks Synced Tables** feature.
- Have the assistant write the configuration to create a synced table pointing from your curated Gold Delta tables directly to Lakebase. This automatically leverages managed Lakeflow Spark Declarative Pipelines to continuously and asynchronously push the Delta table data into Lakebase as queryable, read-only Postgres tables.

Phase 5: Serverless Application Deployment (Streamlit)

To complete the system and serve the data to end-users (coaches or analysts):

- Instruct Claude to write the required Soccermetrics Streamlit web application.
- **Secure Authentication:** Crucially, tell the assistant to use the Databricks SDK for Python to configure **OAuth machine-to-machine (M2M) authentication**. Because the Streamlit app is deployed as a Databricks App, it is assigned a unique service principal identity. The connection pool will automatically generate and rotate OAuth tokens to connect securely to the Lakebase Postgres database without you needing to manage or hardcode any passwords.